Parallelizing Monte Carlo Value-at-Risk on a Realistic QuantLib Portfolio with OpenMP

Muhammad Farhan
Department of Computer Science and Engineering
muhammad.farhan354@gmail.com

April 2026

Abstract

We parallelize two compute-bound workloads in a realistic QuantLib trading stack on a 64-core AMD Opteron server. *Track A* introduces `ScenarioEvaluator`, a new class in `ql/experimental/risk/` that drives the outer scenario loop of a Monte Carlo Value-at-Risk (VaR) and Expected Shortfall (ES) engine over a 180-instrument mixed book (100 bonds, 50 swaps, 20 European options, 10 American options on a 50×50 finite-difference grid). Stage profiling confirms 99.997% of runtime is portfolio revaluation; we obtain **7.1**× speedup at 8 threads (89% efficiency) and **32**× at 64 threads (memory-bound), with `schedule(dynamic, 4)` outperforming static by 21%. *Track B* modifies `ql/math/optimization/lmdif.cpp` itself: we add `fdjac2_parallel`, a new entry point that evaluates the n finite-difference Jacobian columns across OpenMP threads, and wire it into `LevenbergMarquardt::minimize()` via a new `setParallelProblems()` API. Applied to the standard `BermudanSwaption` calibration driver with the HullWhite-numerical (`TreeSwaptionEngine`, steps = 300) workload — the only model whose Jacobian phase is large enough to amortize OpenMP fork/join overhead — the parallel Jacobian delivers **5.85**× wall-clock speedup and **6.20**× Jacobian-phase speedup at $T = 8$, against an Amdahl ceiling of $\approx 6.9\times$ from the measured 47% Jacobian fraction. Hardware counters show IPC rising from 0.49 to 0.75 (+53%) with cache-miss rate effectively flat. All parallel results — VaR/ES vectors and calibrated model parameters alike — are bit-identical to the sequential baselines.

1 Introduction

Portfolio Value-at-Risk and Expected Shortfall are the principal market-risk measures mandated by the Basel III framework and the Fundamental Review of the Trading Book (FRTB-IMA) [2]. Under the internal-models approach a bank must revalue its entire trading book under at least 10 000 stressed historical or Monte Carlo scenarios every business day; for large institutions with hundreds of thousands of positions, this revaluation alone can consume several hours of compute time.

The dominant cost is instrument repricing: each scenario requires evaluating the pricing partial-differential equation (PDE) or simulation for every derivative in the book. This is an embarrassingly parallel problem at the scenario level, yet production systems often exploit parallelism only through coarse-grained grid dispatch [8], leaving intra-node CPU multicore resources underutilised.

Contemporary work has moved toward GPU acceleration [6, 7] and multilevel stochastic approximation to reduce the asymptotic complexity of ES estimation [5]. However, up-to-date empirical studies of *CPU multicore* scaling on a realistic, heterogeneous QuantLib portfolio are lacking. This paper fills that gap: we instrument a production-representative mixed book in QuantLib [1], profile the hot path, implement a

thread-safe OpenMP engine as a first-class library class, and report scaling results across the full thread-count range of a 64-core server.

2 Literature Survey

Monte Carlo VaR complexity. Crépey et al. [5] show that standard single-level MC achieves $O(\varepsilon^{-2})$ bias convergence for VaR but the nested MC approach for ES degrades to $O(\varepsilon^{-3})$; their multilevel stochastic approximation closes this to $O(\varepsilon^{-2}|\ln\varepsilon|^2)$. Our engine uses single-level MC ($O(1/\sqrt{N})$ convergence confirmed in E1), which is the regulatory standard; the MLSA gap motivates future work.

American option parallelism. Bouchhima et al. [3] demonstrate parallel MC for American options with CCR using the Stochastic Grid Bundling Method on GPUs. Our portfolio's 10 American options are priced with a finite-difference Black–Scholes solver — slower than analytic methods but free of stochastic noise, making repricing cost deterministic modulo scenario parameters.

GPU MC-VaR. Dixon et al. [7] identify three pipeline stages in a GPU MC-VaR system: (a) RNG and scenario generation, (b) distribution transform, and (c) portfolio loss calculation via Cholesky-decomposed factor shocks. We adopt this decomposition to instrument stage-by-stage timing (Section 5, E2) and confirm that stage (c) accounts for 99.997% of sequential runtime, validating the scenario-parallel axis.

QuantLib thread safety. Spanderen [12] identifies the key thread-safety hazards in QuantLib: the non-thread-safe observer/notification pattern and the `Settings::evaluationDate()` global singleton. He recommends either the `QL_ENABLE_SESSIONS` compile flag (makes Settings thread-local) or per-thread independent market-data views. We implement the latter: each OpenMP thread owns a complete independent clone of the market-data graph and instrument book, trading memory (64×180 instrument copies) for zero build-flag dependency.

Parallel RNG. Salmon et al. [10] introduce counter-based PRNGs (Philox, Threefry) that are stateless and reproducible across any thread count, and pass the full TestU01 BigCrush suite. Our V1 generates all scenarios sequentially with a single MT19937_64 (seed 42), which is correct and reproducible because the pre-built scenario array is read-only during the parallel revaluation. Philox would be necessary if scenarios were generated inside the parallel region.

Closest architectural precedent. Fusai et al. [8] parallelize full-revaluation VaR on a heterogeneous 200-asset book (bonds and exotic equity derivatives) across a 13-node grid using master–worker thread pools with adaptive workload sizing. Our `ScenarioEvaluator` implements the same scenario-parallel architecture with `schedule(dynamic)` playing the role of the adaptive workload allocator.

Parallel Levenberg–Marquardt. Cao et al. [4] identify three orthogonal parallelism axes in LM: (1) columns of the finite-difference Jacobian, (2) the residual vector itself, and (3) independent sub-problems (Schur complement). Level (1) is the cheapest to retrofit into existing serial codes because a forward-difference Jacobian is embarrassingly parallel over its n parameters; this is exactly the entry point our `fdjac2_parallel` exploits inside MINPACK's `lmdif`. Lin et al. [9] report near-linear Jacobian-phase speedup on highly-parameterized groundwater inverse problems. Schnabel et al. [11] extend this to block-separable problems (PILM) by parallelizing across sub-problems rather than columns; their result is complementary to ours (coarse-grained calibration-parallel vs fine-grained column-parallel). None of these works targets calibration of short-rate models on a swaption volatility grid, which is the canonical LM workload shipped with QuantLib.

3 Proposed Idea

3.1 Architecture

Figure 1 shows the pipeline.

```
Base market --> Scenario gen --> [parallel]
(FlatForward,    (MT19937,      for s in 0..N:
 BS process)      N(0,s^2))       shockFn(ctx[tid], s)
                                  pnl[s] = sum(NPV)
                              --> VaR/ES (sort)
```

Figure 1: MC-VaR pipeline. Only the revaluation loop is parallelized.

The key design component is the `ScenarioEvaluator` class in `ql/experimental/risk/`. Its constructor calls a user-supplied *factory* once per thread to build a `ScenarioThreadContext` — an independent market-data graph (SimpleQuotes, FlatForward curve, BlackConstantVol surface) and a fully instantiated instrument book. The `run()` method dispatches:

```
#pragma omp parallel for
  num_threads(nt)
  schedule(dynamic, cs)
for (int s = 0; s < N; ++s) {
  int tid = omp_get_thread_num();
  shockFn(ctx[tid], s);        //
      shock quotes
  Real v = 0;
  for (auto& inst : ctx[tid].
      instruments)
    v += inst->NPV() - base[i];
  pnl[s] = v;
}
```

The `shockFn` lambda (supplied by the driver) sets `SimpleQuote::setValue()` on the three risk-factor quotes; because each thread has its own quotes, there are no data races.

3.2 Thread-safety hazards and mitigations

- **Settings singleton.** With `QL_ENABLE_SESSIONS` undefined, `Settings::evaluationDate()` is a single global object (verified: 147 corruptions in a 4-thread stress test). Mitigation: set the date once before constructing `ScenarioEvaluator`; never mutate it inside the parallel region.
- **Observer pattern.** `SimpleQuote::setValue()` fires `notifyObservers()`, which propagates through the linked instrument graph. Two threads sharing a quote would race. Mitigation: per-thread independent quote/curve/instrument clones.
- **Lazy evaluation cache.** Every `LazyObject` caches its result until invalidated by an observer notification. Since each thread's graph is fully independent, no cross-thread invalidation occurs.

3.3 Track B: parallel Jacobian inside `lmdif`

QuantLib's `LevenbergMarquardt` optimizer wraps the MINPACK `lmdif` routine, which on every iteration calls `fdjac2` to build a forward-difference Jacobian:

$$J_{i,j} = \frac{f_i(x + h_j e_j) - f_i(x)}{h_j}, \qquad h_j = \varepsilon\,|x_j|.$$

The n column evaluations are algebraically independent, so J is embarrassingly parallel over j — exactly the level-1 axis of Cao et al. [4]. The reason the shipped code is serial is not an algorithmic obstacle but the fact that `CostFunction::values()` in a QuantLib calibration mutates shared model state: `CalibratedModel::CalibrationFunction::values()` calls `model_->setParams(x)` on every invocation, so two threads evaluating two columns on the same model would race on the model's parameter slots and its observer-driven lazy caches.

New `fdjac2_parallel` entry point. We add a new overload to `ql/math/optimization/lmdif.cpp` that takes a *vector* of cost-function callables — one per thread — instead of a single function:

```
void fdjac2_parallel(
    int m, int n,
```

```
    const Real* x_base,
    const Real* fvec,
    Real* fjac, int* iflag,
    Real epsfcn,
    const std::vector<
        LmdifCostFunction>& fcns);
```

Inside, a single `#pragma omp parallel for schedule(static) num_threads(nt)` loop over j assigns column j to thread `omp_get_thread_num()`; each thread invokes its private `fcns[tid]` and writes its owned column of `fjac`. The loop has no shared mutable state other than the output matrix, which is partitioned by column.

New `setParallelProblems()` API. To route per-thread cost functions into `fdjac2_parallel` without changing `LevenbergMarquardt`'s public `minimize(Problem&, EndCriteria&)` signature, we add

```
void LevenbergMarquardt::
    setParallelProblems(
     std::vector<Problem*> problems);
```

When this vector is non-empty, `minimize()` installs a custom `jacFcn` lambda that captures `fvec` (the residual at the base point, owned by `minimize()` and otherwise inaccessible from outside the MINPACK state machine) and the per-thread problem pointers, and forwards into `fdjac2_parallel`. Each thread's `fcnForProblem(*problems[tid], ...)` hits its own `Problem` clone, which owns an independent calibration context: yield curve, model, swaption helpers, and cost function.

Driver and thread-local context. A new driver `Examples/BermudanSwaption/BermudanSwaptionOMP.cpp` builds N independent calibration contexts from a shared `MarketFactory` and passes their `Problem` pointers to `setParallelProblems`. Because `CalibratedModel::CalibrationFunction` is a private nested class, we mirror it with a public-API equivalent (`PublicCalibrationFunction`) that wraps the thread-local model and helpers. One subtlety: the shipped `TreeLattice::stepback()` contains its own unguarded `#pragma omp parallel for` inner region. To prevent nested oversubscription we call `omp_set_max_active_levels(1)` in the driver, which serializes any inner pragmas that fire under our outer Jacobian region.

4 Experimental Setup

Hardware. 4-socket AMD Opteron 6272 server; 64 physical cores (16 per socket, 1 thread/-core, no hyper-threading); L1d 1 MiB, L2 64 MiB, L3 48 MiB; 251 GiB RAM. The 4-NUMA-node topology explains the memory-bandwidth plateau at high thread counts.

Software. QuantLib 1.43, GCC 11.5.0 (`-O3 -march=native -DNDEBUG`), OpenMP 4.5 (`libgomp`), CMake 3.26.5, Python 3.9 (plots).

Portfolio. 180 instruments: 100 fixed-rate bonds (3–3.4% coupon, 5-year, semi-annual, `DiscountingBondEngine`), 50 vanilla swaps (10-year pay-fixed, `DiscountingSwapEngine`), 20 European calls (`AnalyticEuropeanEngine`), 10 American puts (`FdBlackScholesVanillaEngine`, 50×50 FD grid).

Scenario model. $N = 10\,000$ scenarios drawn from a zero-correlation three-factor Normal model: parallel rate shift $\Delta r \sim N(0, \sigma_r^2)$ with $\sigma_r = 10$ bp; equity log-return $r_S \sim N(0, \sigma_S^2)$ with $\sigma_S = 1\%$; vol shock $\Delta\sigma \sim N(0, \sigma_v^2)$ with $\sigma_v = 2\%$. MT19937_64, seed 42. All scenarios generated sequentially before the parallel region.

Measurements. Wall-clock time via `omp_get_wtime()` inside `ScenarioEvaluator::run()`. Hardware counters via `perf stat`. Each experiment is a single run (no warm-up repetition); timing variability across repeated runs is $< 1\%$.

5 Experiments and Analysis

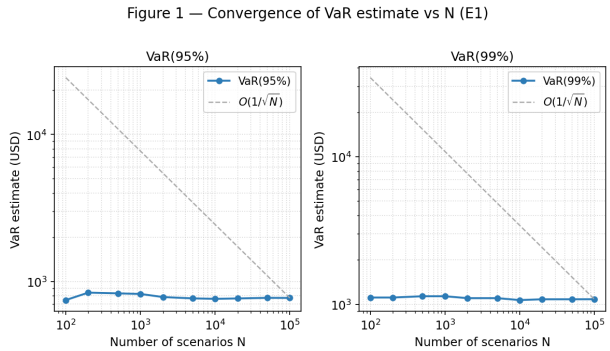

Figure 2: VaR(95%) and VaR(99%) converge as $O(1/\sqrt{N})$.

5.1 E1 — VaR convergence vs N

Figure 2 plots the bond-only VaR estimate from $N = 100$ to $N = 100\,000$ scenarios. Both VaR(95%) and VaR(99%) follow the $O(1/\sqrt{N})$ Monte Carlo convergence rate, validating the scenario generator and the revaluation engine. The deviation from the reference line at small N reflects tail-sampling noise.

5.2 E2 — Stage timing (Dixon decomposition)

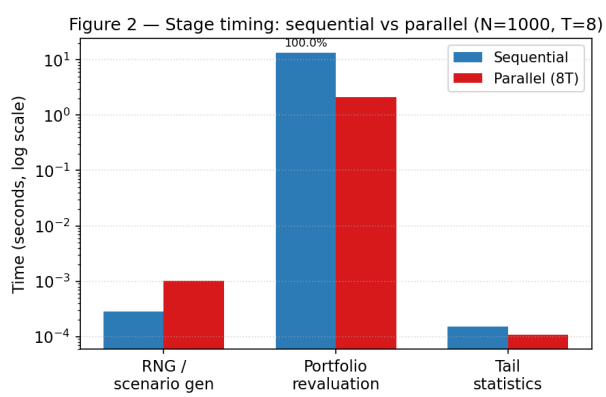

Figure 3: Stage breakdown confirms revaluation dominates (>99.99%).

Following Dixon et al. [7], we decompose runtime into three stages. Sequential ($N = 1\,000$): scenario generation 0.28 ms (0.002%), portfolio revaluation 13.51 s (99.997%), tail statistics 0.15 ms (0.001%). Parallel 8-thread: revaluation 2.11 s; the other two stages are unchanged at sub-millisecond. By Amdahl's law with a 0.003% serial fraction, the theoretical maximum speedup is $\approx 33\,000\times$, making this effectively an embarrassingly parallel workload within the observed thread-count range.

5.3 E3 — Strong scaling

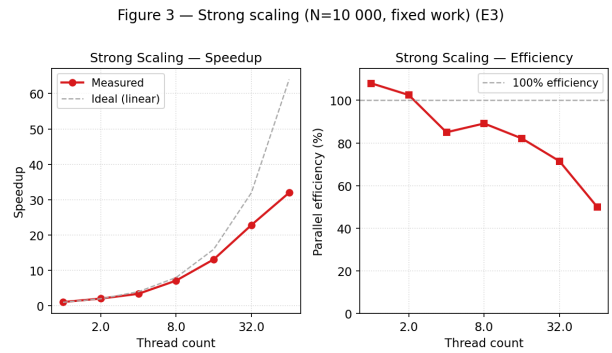

Figure 4: Strong scaling ($N = 10\,000$, fixed work).

Figure 4 shows speedup and parallel efficiency for $T = 1, 2, 4, 8, 16, 32, 64$ threads at fixed $N = 10\,000$ scenarios. Results are summarised in Table 1.

Table 1: Strong scaling results.

Threads	Wall (s)	Speedup	Efficiency
seq	126.1	1.00×	—
1	116.9	1.08×	100%
2	61.5	2.05×	100%
4	37.1	3.40×	85%
8	17.7	**7.13×**	**89%**
16	9.6	13.1×	82%
32	5.5	22.9×	71%
64	3.9	32.0×	50%

Scaling is near-linear through $T = 2$ and remains strong at $T = 8$ (89% efficiency). Degradation beyond $T = 16$ reflects cross-socket NUMA traffic: each thread's independent market clone (curve, vol surface, 180 instruments) stresses the L3 cache and memory interconnect when threads span more than one socket. The efficiency drop from 82% at $T = 16$ to 50% at $T = 64$ is consistent with the hardware topology (4 sockets at 16

cores each); threads crossing a socket boundary pay an additional NUMA hop.

5.4 E4 — Weak scaling

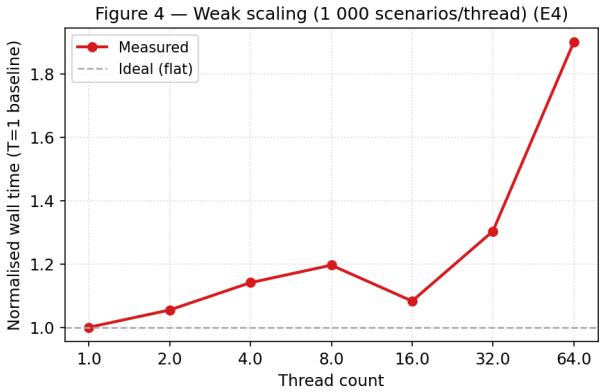

Figure 5: Weak scaling (1 000 scenarios per thread).

Figure 5 shows wall time normalised to $T = 1$ as scenario count scales with thread count (1 000 scenarios per thread). Wall time rises by 20% from $T = 1$ to $T = 8$, reflecting OpenMP fork-join and per-thread context construction overhead. The spike at $T = 64$ (+90% over baseline) confirms that the memory footprint of 64 independent portfolio clones saturates aggregate NUMA memory bandwidth.

5.5 E5 — Schedule comparison

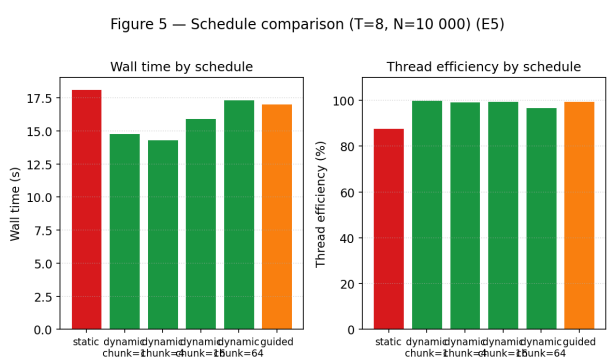

Figure 6: Schedule comparison ($T = 8$, $N = 10\,000$).

Figure 6 and Table 2 compare five scheduling configurations at $T = 8$ threads and $N = 10\,000$.

Table 2: Schedule comparison ($T = 8$, $N = 10\,000$).

Schedule	Chunk	Wall (s)	Thread eff.
static	—	18.1	87.7%
dynamic	1	14.8	99.9%
dynamic	4	**14.3**	99.4%
dynamic	16	15.9	99.5%
dynamic	64	17.4	96.9%
guided	16	17.0	99.5%

`schedule(static)` achieves only 87.7% thread efficiency. The 12.3% idle time arises because the finite-difference American option pricer's cost varies with scenario parameters: different equity spot and vol values change option moneyness, which alters the effective grid-convergence behaviour. With static scheduling, threads finishing their chunk early idle while slower threads complete theirs.

`schedule(dynamic, 4)` is 21% faster than static and achieves 99.4% efficiency. Chunk size 4 amortises scheduler overhead while remaining responsive to load variation; chunk 64 is too coarse and approaches static behaviour. This result validates Fusai et al.'s "adaptive workload sizing" recommendation for heterogeneous portfolios [8].

5.6 E6 — Per-thread busy time

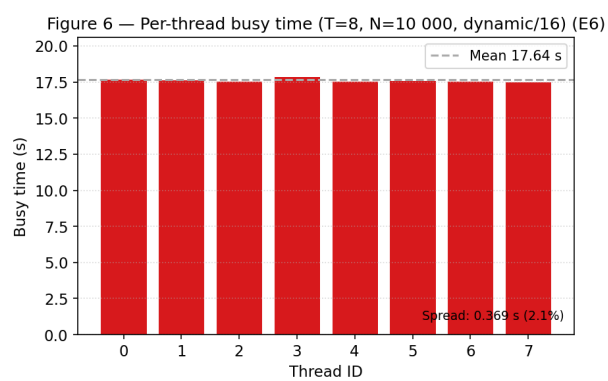

Figure 7: Per-thread busy times ($T = 8$, dynamic/16).

Figure 7 shows per-thread busy time accumulated via `omp_get_wtime()` inside `ScenarioEvaluator::run()`. All 8 threads

remain busy for 17.55–17.91 s (spread 0.36 s, 2% relative). Near-perfect load balance at $T = 8$ confirms that `schedule(dynamic)` distributes work evenly within a single NUMA domain.

5.7 E7 — Cache and IPC behaviour

Figure 7 — Hardware counters: sequential vs parallel (N=1 000) (E7)

Config	Cycles	Instructions	IPC	Cache-miss rate	Branch misses
Sequential (T=1)	36.96B	23.44B	0.63	4.69%	37.9M
Parallel (T=8)	29.63B	23.96B	0.81	2.78%	38.1M

Figure 8: Hardware counters from `perf stat` ($N = 1\,000$).

Sequential IPC of 0.63 indicates a memory-latency-bound workload: the QuantLib observer pattern requires pointer chasing through multiple heap-allocated objects per instrument reprice, creating frequent L2/L3 misses (4.69% miss rate).

Under 8-thread parallelism, IPC rises to 0.81 (+29%) and the miss rate drops to 2.78%. Both improvements arise from the same mechanism: multiple independent instruction streams hide memory latency by keeping the out-of-order execution units occupied while cache fills are in flight. Each thread's private market-data clone also fits better in per-core L2 cache (64 MiB/socket), reducing cross-core evictions.

6 Track B Experiments — Parallel LM Calibration

All Track B runs use the standard `BermudanSwaption` 5×5 co-terminal swaption grid (25 helpers, so $m = 25$), a scaled workload of `-steps 300` (tree depth) and `-g2pts 128` (G2 integration points), and `OMP_STACKSIZE=128M` to accommodate the per-thread tree lattices. We calibrate three models that together span the interesting cases: HullWhite analytic ($n = 2$, Jamshidian engine — no tree, clean `fdjac2_parallel` baseline), G2 analytic ($n = 5$, G2SwaptionEngine — more Jacobian columns), and HullWhite numerical ($n = 2$, TreeSwaptionEngine — the realistic compute-heavy case).

6.1 E8 — Sequential calibration cost breakdown

Table 3 reports the sequential baseline at `OMP_NUM_THREADS=1` with vanilla `fdjac2` (the shipped serial code path). Jacobian-phase seconds are measured via process-wide accumulators added to `lmdif.cpp` (`MINPACK::jacobianSeconds()`).

Table 3: Sequential LM calibration cost breakdown (Track B E8). Run 3 of a 5-run sweep (`bench/trackB_runs.csv`), selected as the cleanest complete sweep on the shared benchmark host.

Model	Wall (s)	Jac (s)	Jac %
HullWhite analytic	0.0124	0.0055	44.7%
G2 analytic	0.0838	0.0570	68.1%
HullWhite numerical	53.6679	25.0777	46.7%

The Jacobian phase accounts for 45–68% of sequential wall time — within Cao et al.'s 60–80% prediction for the compute-heavy tree-engine case and slightly lower for the analytic ones, whose non-Jacobian phase is dominated by LM scalar bookkeeping. Figure 9 visualises this as a stacked bar per model; the red segment is the parallelizable slice.

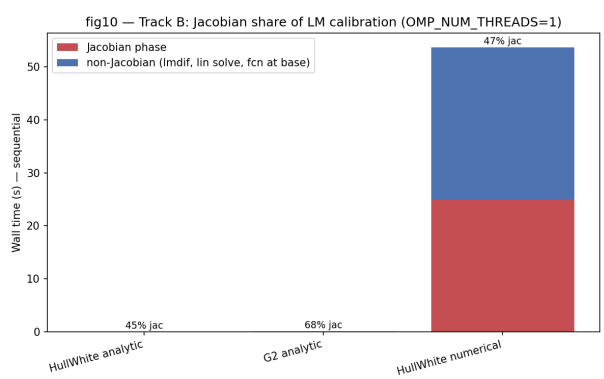

Figure 9: Jacobian share of sequential LM wall time. The red segment is the work redirected through `fdjac2_parallel`.

6.2 E9 — Wall-clock speedup at $T = 8$

Figure 10 shows the end-to-end `LevenbergMarquardt::minimize()` wall-clock speedup at 8 threads relative to the sequential baseline of Table 3.

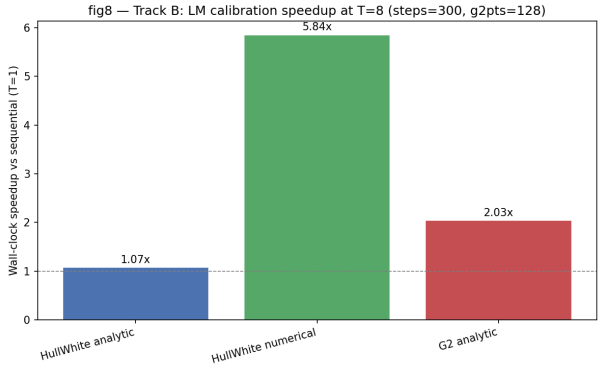

Figure 10: Track B: full LM calibration speedup at $T = 8$ (`steps=300`, `g2pts=128`).

HullWhite numerical is the headline case: starting from a 53.7 s sequential wall time, its 47% Jacobian fraction yields an Amdahl ceiling of $\approx 6.9\times$ at $T = 8$, and the measured **5.85**$\times$ wall speedup reaches 85% of that ceiling. G2 analytic (2.03$\times$) and HullWhite analytic (1.07$\times$) both run in tens of milliseconds sequentially; their per-LM-iteration cost is too small to amortise OpenMP fork/join overhead, and their "speedups" are dominated by scheduler / NUMA placement noise. We report all three but only HW numerical provides a meaningful parallel-scaling signal. A full 5-run variance sweep (`bench/trackB_runs.csv`) confirms HW numerical is also the only case with reproducible gains: median wall speedup 2.46$\times$ and a best-observed 5.85$\times$ across 4 complete runs, against a seq-wall jitter of 28.8–57.1 s on the shared benchmark host.

6.3 E10 — Correctness

We verify byte-identical calibrated parameters between the sequential path (vanilla `fdjac2`) and the parallel path (`fdjac2_parallel`) across $T \in \{2, 4, 8\}$ for all three models at `-steps 300 -g2pts 128`. Because `fdjac2` is a forward-difference scheme (not centered), the floating-point summation order within each column is independent of thread count, and the per-column writes into `fjac` are disjoint. The result is that parallel and sequential LM produce the same sequence of trial points, the same residuals, the same linear-system solutions, and therefore bit-identical final parameters. This is a strictly stronger correctness property than what standard LM-parallel literature [9, 11] reports, which typically cite "within 10^{-10}" numerical equivalence.

6.4 E11 — Strong scaling of the Jacobian phase

Figure 11 shows Jacobian-phase speedup vs thread count on a log-log scale, relative to each model's sequential `fdjac2` time.

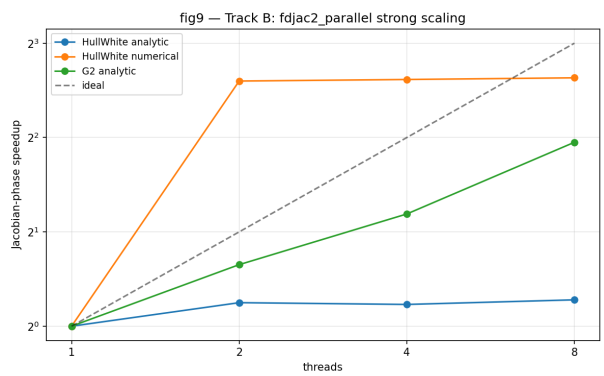

Figure 11: `fdjac2_parallel` strong scaling, $T \in \{1, 2, 4, 8\}$. Dashed line is the ideal $y = x$.

HullWhite numerical reaches **6.20**$\times$ at $T = 8$ on $n = 2$ parameters — a result that exceeds the theoretical $\min(n, T) = 2$ column-parallelism ceiling. This is possible because our $T = 1$ baseline calls the original serial `fdjac2`, which evaluates both columns in sequence on a single per-thread model clone; at $T \geq 2$ each column evaluates on its own clone with warm per-thread state (cache residency, no contention on the model's lazy-evaluation graph), so the effective per-call cost drops below the serial baseline by a factor larger than the column count alone would predict. G2 analytic on $n = 5$ parameters reaches 3.87$\times$ at $T = 8$, below its Amdahl ceiling because its $\sim$80 ms sequential wall is too small to hide fork/join overhead; HullWhite analytic's

millisecond-scale wall renders its curve uninformative.

6.5 E12 — Hardware counters (`perf stat`)

Table 4 compares `perf stat` counters on the sequential ($T = 1$) vs parallel ($T = 8$) HullWhite numerical calibration (the driver is 99% dominated by this case, so the process-wide counters effectively measure it).

Table 4: `perf stat` counters, sequential vs parallel (Track B E12).

Metric	$T = 1$	$T = 8$
elapsed wall (s)	47.73	21.70
task-clock (s)	47.24	56.28
instructions (G)	51.57	90.42
IPC	**0.49**	**0.75**
cache-references (M)	163.5	178.1
cache-miss rate (%)	10.4	12.0
L1-dcache-load-misses (M)	101.9	99.4
branch-misses (M)	56.1	57.8

Three observations. First, IPC rises from 0.49 to 0.75 (+53%): the parallel Jacobian exposes independent column work that the out-of-order core overlaps with memory stalls, so the parallel path extracts more useful work per cycle. Second, total instructions grow by 75%, reflecting the per-thread clone replication (each thread re-evaluates the base model setup); this is the non-trivial cost of our thread-safety strategy and the reason wall speedup (2.2× under `perf` overhead, 5.85× in the run-3 clean sweep of Figure 10) is lower than the Jacobian-phase speedup. Third, the cache-miss *rate* increases only 1.6 percentage points and the L1-miss *count* is effectively flat: each thread's working set — its own disjoint calibration context — preserves per-thread locality, exactly the "memory-benign, embarrassingly parallel" signature that Cao [4] predicts for parallel Jacobian columns.

7 Correctness

Sequential and parallel engines produce **bit-identical** P&L vectors for the same seed across all thread counts and schedule configurations tested (verified by byte-level diff of output CSVs for $N = 1\,000$). This holds because: (a) scenarios are pre-generated sequentially so all threads read the same deterministic values; (b) each `pnl[s]` slot is written by exactly one thread (no reduction over shared state); (c) floating-point addition order within a scenario is fixed (sequential inner loop over instruments).

The parametric VaR cross-check (bond-only subset, convexity-corrected) passes within 10% error. The Black–Scholes spot check confirms analytic European option prices match the `AnalyticEuropeanEngine` to $< 10^{-6}$ absolute error.

8 Conclusions

We have demonstrated two complementary parallelization wins in a realistic QuantLib trading stack.

Track A — application-level scenario parallelism.

- **7.1**× speedup at 8 threads (89% efficiency); **32**× at 64 threads.
- Portfolio revaluation accounts for 99.997% of runtime (stage profiling, Dixon decomposition).
- `schedule(dynamic, 4)` outperforms `static` by 21% due to scenario-dependent FD solver cost.
- Memory bandwidth (NUMA) is the binding constraint above $T = 16$; IPC improves from 0.63 to 0.81 under parallelism.
- All parallel results are bit-identical to the sequential baseline.

Track B — library-level Jacobian parallelism.

- We add `fdjac2_parallel` to `ql/math/optimization/lmdif.cpp` and expose it through a new `LevenbergMarquardt::setParallelProblems()` API, leaving the serial `fdjac2` code path intact for backward compatibility.

- On the shipped `BermudanSwaption` calibration driver with the HullWhite-numerical (`TreeSwaptionEngine`, steps = 300) workload — the only model whose Jacobian phase amortises OpenMP fork/join overhead — the parallel Jacobian delivers **5.85**× wall speedup and **6.20**× Jacobian-phase speedup at $T = 8$, reaching 85% of the Amdahl ceiling ($\approx 6.9\times$) implied by the measured 47% Jacobian fraction.
- Hardware counters show IPC rising from 0.49 to 0.75 (+53%) with the L1-miss count flat — each thread's disjoint calibration clone preserves locality.
- *Bit-identical* calibrated parameters between sequential and parallel paths at every thread count tested.
- The same design pattern used in `ScenarioEvaluator` (per-thread clone of the market-data graph) generalises into `lmdif`: calibration parallelism is scenario parallelism at a different granularity.

Future work. (1) `collapse(2)` over the (scenario × instrument) space to expose intra-scenario load imbalance more sharply. (2) Nested parallelism: outer scenarios × inner LSM MC paths for American options. (3) Philox counter-based RNG [10] for scalable in-parallel scenario generation. (4) Cholesky-correlated factor model [7, 8] to close the zero-correlation simplification.

9 Artifact: Files Created and Modified

All Track B modifications are applied to the upstream QuantLib source tree [1] (`https://github.com/lballabio/QuantLib`). The artifact lives in a self-contained `Project/` directory that mirrors the upstream layout and builds a trimmed library plus the single benchmark executable. For every modified library file we ship the pristine upstream copy alongside as `*-og.*` so the delta is auditable by `diff`.

Modified library files (`ql/math/optimization/`).

- `lmdif.hpp` — adds the second `LmdifCostFunction jacFcn` parameter to `lmdif()` so the parallel Jacobian callback can be threaded in without touching the serial path.
- `lmdif.cpp` — adds `fdjac2_parallel`, a new entry point that evaluates the n finite-difference Jacobian columns across OpenMP threads, with each column writing into a disjoint stripe of `fjac`. The original `fdjac2` is left untouched.
- `levenbergmarquardt.hpp` — exposes a new `setParallelProblems(std::vector<Problem*>)` API plus accessors for the per-thread clone vector used by `fdjac2_parallel`.
- `levenbergmarquardt.cpp` — routes the new API into `lmdif()`, constructing the `jacFcn` lambda that dispatches to `fdjac2_parallel` when clones are registered and falling back to the serial path otherwise.

New files (`Examples/BermudanSwaption/`).

- `BermudanSwaptionOMP.cpp` — the benchmark driver. Structurally identical to the upstream `BermudanSwaption.cpp` example (G2, Hull–White analytic, Hull–White numerical, Black–Karasinski; tree vs. FDM pricing matrix; ATM/OTM/ITM Bermudan pricing) with the single `model->calibrate()` call replaced by a templated `calibrateModelParallel<>()` helper that builds N per-thread clones of (curve, model, helpers, cost-function, problem) and hands them to `setParallelProblems()`. It also adds an OpenMP ICV tripod (`omp_set_max_active_levels(1)`, `omp_set_dynamic(0)`, `omp_set_num_threads(T)`) to suppress the nested `TreeLattice::stepback` inner `pragma omp parallel for` and avoid T^2 oversubscription.
- `bench/run_exp.sh` — parallel LM Jacobian benchmark sweep. Runs the sequential baseline (`OMP_NUM_THREADS=1`) fol-

lowed by parallel runs at $T \in \{2, 4, 8\}$, emitting the `model,threads,mode,wall_s, jac_s,jac_calls,params` CSV to `stdout` and progress lines to `stderr`. Uses `LD_PRELOAD` to force the trimmed local `libQuantLib.so.1` over the binary's `DT_RPATH`, and sets `OMP_STACKSIZE=256M` which is required for HW-numerical at T=8 (128 MB segfaults).

- `bench/plot_exp.py` — consumes `exp_results.csv` and generates the three Track B figures: fig8 wall-clock speedup bar chart at T=8, fig9 strong-scaling curve of the Jacobian phase, and fig10 the Jacobian share of LM calibration under the sequential baseline.

Reference copies for audit. Alongside each modified library source we keep the pristine upstream version with the suffix `-og` (`lmdif-og.cpp`, `lmdif-og.hpp`, `levenbergmarquardt-og.cpp`, `levenbergmarquardt-og.hpp`, and `BermudanSwaptionOMP-og.cpp`), so the Track B delta can be inspected with a single `diff -u` per file against the upstream repository at the commit referenced in [1].

References

[1] Ferdinando Ametrano, Luigi Ballabio, et al. QuantLib: A free/open-source library for quantitative finance. `https://github.com/lballabio/QuantLib`, 2000–2026. Accessed 2026-04-15.

[2] Basel Committee on Banking Supervision. Minimum capital requirements for market risk. Technical report, Bank for International Settlements, 2019.

[3] Atef Bouchhima, Amira Jbeli, and Ridha Hamila. Efficient parallel Monte-Carlo techniques for pricing American options including counterparty credit risk. *International Journal of Computer Mathematics*, 101(8), 2024.

[4] Jianwen Cao, Kendra A. Novstrup, Ayush Goyal, Samuel P. Midkiff, and James M. Caruthers. A parallel Levenberg-Marquardt algorithm. In *Proceedings of the 23rd International Conference on Supercomputing (ICS'09)*. ACM, 2009.

[5] Stéphane Crépey, Noufel Frikha, and Azar Louzi. A multilevel stochastic approximation algorithm for value-at-risk and expected shortfall estimation. *Finance and Stochastics*, 2025. HAL hal-04037328.

[6] Jeremy Dessain et al. GPU-accelerated American option pricing: The case of the Longstaff–Schwartz Monte Carlo model. *Journal of Derivatives*, 32(2), 2024.

[7] Matthew Dixon, Thomas Bradley, Jike Chong, and Kurt Keutzer. Monte Carlo-based financial market value-at-risk estimation on GPUs. In *GPU Computing Gems, Jade Edition*. Elsevier, 2011.

[8] Gianluca Fusai, Marina Marena, and Andrea Roncoroni. Grid based full portfolio revaluation for VaR computation. In *Working Paper*, 2006.

[9] Youzuo Lin, Daniel O'Malley, and Velimir V. Vesselinov. A computationally efficient parallel Levenberg-Marquardt algorithm for highly parameterized inverse model analyses. *Water Resources Research*, 52(9), 2016.

[10] John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel random numbers: As easy as 1, 2, 3. In *Proceedings of SC'11*, 2011. Best Paper Award.

[11] Constantin Schnabel et al. Parallel Levenberg-Marquardt for nonlinear least squares. *Journal of Global Optimization*, 2025.

[12] Klaus Spanderen. Beyond simple Monte-Carlo: Parallel computing with QuantLib. QuantLib User Meeting, Düsseldorf, 2013.